

Manual Completo de Replicación: Sistema MicroGas (Control DJI + App Celular)

GasLab — Monitoreo de Gases Atmosféricos con Dron DJI Mavic 3T

Revisión documental: 2026-09-30. Receptor declara versionName 0.4.3 y versionCode 7; DJI y Demo siguen en 0.4.1 (5). Demo incluye ruta y altura sintéticas; el receptor incluye la tercera pestaña Ruta y altura.

Este es el manual principal. Consulta primero la sección de incorporación añadida al final. Los manuales anteriores se conservan en docs/archivo como referencia histórica, no como instrucciones vigentes.

Índice

1. [Visión General del Sistema y Arquitectura de Hardware](#)
2. [Variables Físicas del Sensor SEN66 y Exclusión de SO₂](#)
3. [Protocolos de Comunicación y Estructura de Datos](#)
 - 3.1. Pi 5 → Mavic 3T → RC Pro (PSDK v1 de 32 B y v2 de 64 B)
 - 3.2. RC Pro → Celular (Bluetooth Clásico RFCOMM / NDJSON v1)
 - 3.3. Estructura de Archivos CSV (24 Columnas en Control, 16 Columnas en Celular)
 - 3.4. Límites de la validación
4. [App 1: MicroGas para DJI RC Pro Enterprise \(app_dji\)](#)
 - 4.1. Requisitos y Herramientas de Desarrollo
 - 4.2. Configuración de Gradle, Dependencias y Flavors (demo y dji)
 - 4.3. AndroidManifest y Permisos Específicos
 - 4.4. Componentes Clave y Arquitectura de Software
 - 4.5. Pasos de Compilación, Instalación y Variantes de APK
5. [App 2: MicroGas Receptor para Celular Android \(app_receptor\)](#)
 - 5.1. Requisitos y Herramientas de Desarrollo
 - 5.2. Componentes clave
 - 5.3. Interfaz y zoom
 - 5.4. Compilación y APK
6. [Guía de Integración y Prueba Extremo a Extremo](#)
 - 6.1. Prueba Local sin Hardware (Modo Simulado / Demo)
 - 6.2. Prueba en Banco de Trabajo con Hardware Real
 - 6.3. Prueba de convivencia con DJI Pilot 2
7. [Resolución de Problemas Frecuentes \(Troubleshooting\)](#)
8. [Incorporación](#)
9. [Prueba guiada](#)
10. [Interpretar el registro](#)
11. [Análisis CSV y mapas](#)
12. [Hardware y desarrollo](#)
13. [Guía de Desarrollo con Android Studio y Extensión de Features](#)
 - 13.1. Cómo funciona Android Studio (Guía paso a paso para quien nunca lo ha usado)
 - 13.2. Mapa de desarrollo: ¿Qué archivo modifico o creo para cada feature?
 - 13.3. Compilación automatizada mediante Makefile (make apks)

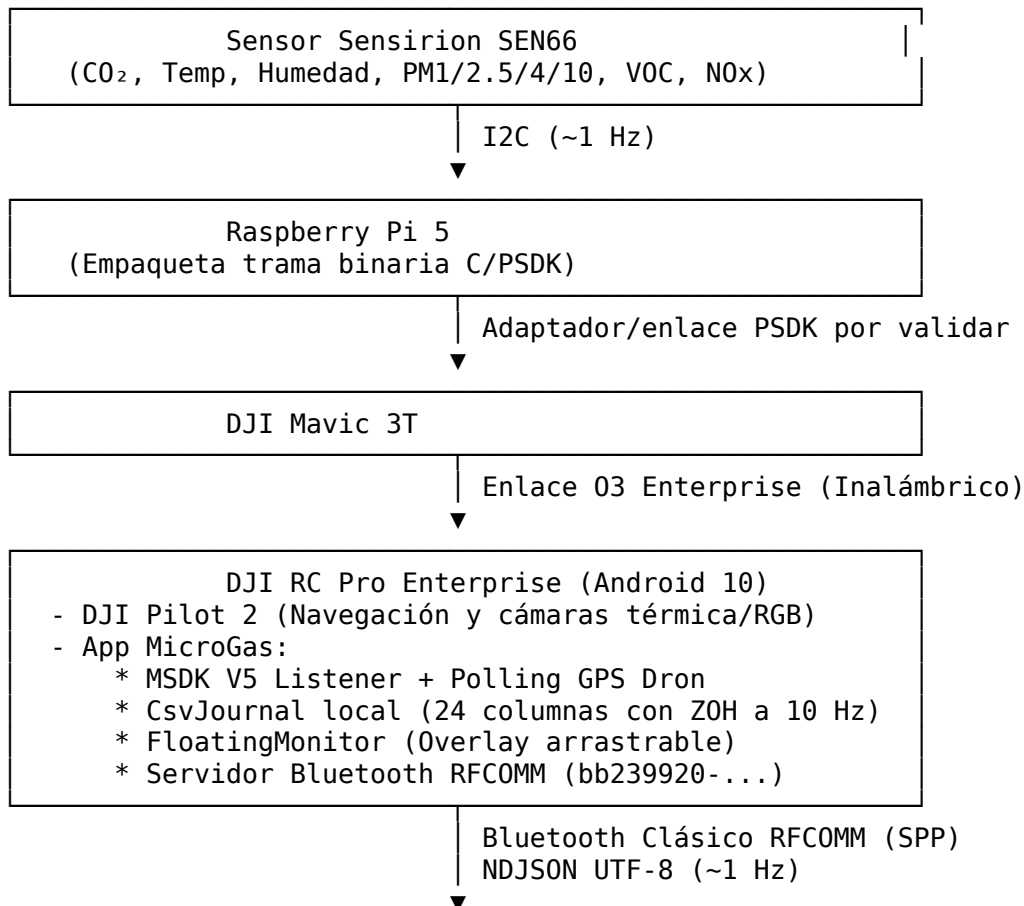
14. [Receptor 0.4.3: Home, máximos y CSV](#)
 15. [Cómo editar menús e interfaz en Android Studio](#)
 - 15.1. Orientarse en Android Studio y encontrar el archivo correcto
 - 15.2. Editar la GUI: botones, diálogos y pestañas (con ejemplos reales)
 - 15.3. Editar la lógica interna: ViewModel y estado
 - 15.4. Previsualizar cambios sin instalar en el teléfono
 - 15.5. Flujo de trabajo recomendado
-

1. Visión General del Sistema y Arquitectura de Hardware

El sistema **MicroGas** adquiere, visualiza, georreferencia y registra concentraciones de gases y aerosoles en tiempo real mediante una carga útil instalada en un dron DJI Mavic 3 Enterprise / Thermal (Mavic 3T).

El ecosistema está compuesto por dos aplicaciones sincronizadas: 1. **Control DJI RC Pro Enterprise:** corre la app MicroGas, la cual procesa los paquetes nativos de la Raspberry Pi 5 vía DJI Mobile SDK V5 (MSDK), añade las coordenadas GNSS del dron, registra un archivo CSV sincronizado a 10 Hz (Zero-Order Hold), proporciona una ventana flotante (*overlay*) sobre DJI Pilot 2 y retransmite los datos vía **Bluetooth RFCOMM como servidor**. 2. **Celular Android (Estación de Monitoreo Terrestre):** corre la app MicroGas Receptor, la cual se conecta como **cliente Bluetooth clásico RFCOMM** al control, grafica en tiempo real las concentraciones, almacena el histórico multivariable (con zoom y desplazamiento horizontal), permite exportar CSV mediante Storage Access Framework (SAF) y emite alertas visuales y vibratorias por superación de umbrales.

Diagrama de Enlace Físico



- Celular Android (Android 8.0+)
- App MicroGas Receptor:
 - * Cliente RFCOMM con reconexión automática
 - * Pestaña 1: Monitor CO₂ en tiempo real
 - * Pestaña 2: Histórico (CO₂, Temp, Hum, PM...)
 - * Respaldo local y Exportación CSV (16 columnas)

2. Variables Físicas del Sensor SEN66 y Exclusión de SO₂

[!IMPORTANT] **El sensor físico instalado es un Sensirion SEN66.** El SEN66 mide exactamente **9 variables ambientales**: 1. Concentración de dióxido de carbono: **CO₂** (ppm) 2. **Temperatura ambiente** (°C) 3. **Humedad relativa** (% HR) 4. Material particulado fino: **PM1.0** (µg/m³) 5. Material particulado: **PM2.5** (µg/m³) 6. Material particulado: **PM4.0** (µg/m³) 7. Material particulado grueso: **PM10** (µg/m³) 8. Índice de compuestos orgánicos volátiles: **VOC Index** (escala 1-500) 9. Índice de óxidos de nitrógeno: **NOx Index** (escala 1-500)

El SEN66 NO mide dióxido de azufre (SO₂).

Por este motivo, ni la aplicación del control ni la del celular incluyen SO₂ en interfaces, modelos, bases de datos ni archivos CSV.

3. Protocolos de Comunicación y Estructura de Datos

3.1. Pi 5 → Mavic 3T → RC Pro (PSDK v1 y v2)

La app del control soporta de manera transparente ambas versiones del protocolo binario PSDK recibido en PayloadIndexType.UP.

Protocolo v2 de 64 Bytes (Versión 0.3.0 — Nueve Variables) Todos los campos enteros y float32 IEEE-754 usan orden **Little-Endian**:

Offset	Tamaño	Tipo	Campo	Descripción
0-3	4 B	ASCII	magic	Valor constante MGAS (0x4D, 0x47, 0x41, 0x53)
4	1 B	uint8	version	Versión mayor del protocolo: 2 (0x02)
5	1 B	uint8	flags	Flags: 1 (CO ₂ obligatorio y válido)
6-7	2 B	uint16	reserved	Relleno reservado: 0x0000
8-11	4 B	uint32	boot_id	Identificador único generado en cada reinicio del emisor
12-15	4 B	uint32	sequence	Contador monotónico de muestras (admite rollover)
16-23	8 B	int64	uptime	Milisegundos monotónicos de funcionamiento de la Pi
24-27	4 B	float32	co2_ppm	Concentración de CO ₂ (ppm)

Offset	Tamaño	Tipo	Campo	Descripción
28-31	4 B	float32	temperature_c	Temperatura ambiente (°C) o NaN si no está disponible
32-35	4 B	float32	humidity_pct	Humedad relativa (% HR, 0-100) o NaN
36-39	4 B	float32	pm1_0	Masa PM1.0 ($\mu\text{g}/\text{m}^3$) o NaN
40-43	4 B	float32	pm2_5	Masa PM2.5 ($\mu\text{g}/\text{m}^3$) o NaN
44-47	4 B	float32	pm4_0	Masa PM4.0 ($\mu\text{g}/\text{m}^3$) o NaN
48-51	4 B	float32	pm10	Masa PM10 ($\mu\text{g}/\text{m}^3$) o NaN
52-55	4 B	float32	voc_index	Índice VOC (1-500) o NaN
56-59	4 B	float32	nox_index	Índice NOx (1-500) o NaN
60-63	4 B	uint32	crc32	CRC32 estándar (polinomio 0xEDB88320) sobre bytes 0-59

Protocolo Legado v1 de 32 Bytes

- Se conserva retrocompatibilidad total: si el paquete entrante mide 32 bytes y su versión es 1 (bytes[4] == 1), se extrae co2_ppm en el offset 24 y el CRC32 en el offset 28. Los 8 canales opcionales se completan automáticamente como null.

3.2. RC Pro → Celular (Bluetooth Clásico RFCOMM / NDJSON v1)

El control ejecuta un servidor RFCOMM con Serial Port Profile (SPP): - **UUID:** bb239920-bdbc-4d51-8a12-a5351875d891 - **Nombre de Servicio:** MicroGas - **Formato:** NDJSON (línea JSON UTF-8 terminada en \n por cada muestra física nueva a ~1 Hz). - **Control de Congestión:** Canal conflado (Channel<Pair<Long, ByteArray>>(CONFLATED)). Si el celular es lento procesando, se descartan muestras intermedias en la cola para no generar latencia. Muestras en cola > 5 s se descartan automáticamente.

Ejemplo de Paquete JSON Transmitido

```
{
  "v": 1,
  "session": "550e8400-e29b-41d4-a716-446655440000",
  "sequence": 15,
  "received_at_ms": 1790000000123,
  "co2_ppm": 650.250,
  "temperature_c": 24.310,
  "humidity_pct": 58.400,
  "pm1_0": 4.200,
  "pm2_5": 7.150,
  "pm4_0": 9.800,
  "pm10": 11.300,
  "voc_index": 115.000,
  "nox_index": 1.000,
  "origin": "dji",
  "sender_boot": 4,
  "sender_sequence": 99,
  "acquired_uptime_ms": 1000,
  "aircraft_position": {
    "latitude": 9.123456,
    "longitude": -84.987654,
    "source": "dji_aircraft_msd",
    "received_at_ms": 1790000000000,
    "age_at_sample_ms": 123,
    "signal_level": 4
  }
}
```

```
}  
}
```

3.3. Estructura de Archivos CSV

CSV del Control DJI (app_dji — 24 Columnas) El control registra en files/sessions/ con frecuencia desacoplada de 10 Hz (Zero-Order Hold):

fecha_hora_utc,co2_ppm,sen66_new_data,sen66_muestra_utc,sen66_secuencia,latitud,longitud,orig

- sen66_new_data: 1 si es una muestra nueva que acaba de llegar, 0 si es una fila retenida por el Zero-Order Hold.

CSV del Celular Receptor (app_receptor — 16 Columnas) El celular guarda cada muestra física recibida (~1 Hz) y permite exportar mediante Storage Access Framework:

fecha_hora_utc,co2_ppm,temperatura_c,humedad_pct,latitud,longitud,origin,session,sequence,pm1

- Marcas de tiempo en ISO-8601 UTC con milisegundos (yyyy-MM-dd'T'HH:mm:ss.SSS'Z').
 - Punto decimal forzado con Locale.US.
-

3.4. Límites de la validación

El payload v2 contiene 64 bytes por muestra, pero esa cifra no incluye transporte ni prueba ausencia de interferencia con Pilot 2. La carga de Bluetooth, memoria y escritura depende del dispositivo y debe medirse. `fd.sync()` reduce datos pendientes; no garantiza recuperar muestras perdidas o resistir una falla física del almacenamiento. La integración del lector SEN66 en la Pi y la recepción física siguen pendientes de validación en este montaje.

4. App 1: MicroGas para DJI RC Pro Enterprise (app_dji)

4.1. Requisitos y Herramientas de Desarrollo

- **IDE:** Android Studio Ladybug (2024.2+) o superior.
- **JDK:** Java 17 (configurado como Gradle JDK).
- **Android SDK:** `compileSdk 35,minSdk 29` (Android 10), `targetSdk 34`.
- **Arquitectura NDK:** `arm64-v8a`.
- **Dependencias:** DJI MSDK V5 Aircraft 5.18.0, Compose BOM 2024.12.01, Kotlin 2.0.21.

4.2. Configuración de Flavors

El proyecto define dos *Product Flavors*: - **demo (com.gaslab.microgas.demo):** Genera las 9 variables de forma sintética (onda sinusoidal a ~1 Hz). No requiere hardware DJI, ni permisos especiales de vuelo, ni App Key. Ideal para pruebas de interfaz y Bluetooth sin encender el dron. - **dji (com.gaslab.microgas):** Enlaza con las librerías nativas de DJI MSDK V5, escucha el canal PSDK y lee la posición del dron por telemetría. Requiere App Key en `local.properties`: `properties dji.msdk.apiKey=TU_CLAVE_DJI_MSDK_AQUI`

4.3. AndroidManifest y Permisos

- `SYSTEM_ALERT_WINDOW`: Para la ventana flotante sobre DJI Pilot 2.
- `FOREGROUND_SERVICE` y `FOREGROUND_SERVICE_CONNECTED_DEVICE`: Adquisición continua en segundo plano.
- `BLUETOOTH_CONNECT`, `BLUETOOTH`, `BLUETOOTH_ADMIN`: Servidor Bluetooth RFCOMM.

- `ACCESS_FINE_LOCATION`, `ACCESS_COARSE_LOCATION`: Permisos solicitados por la integración MSDK. El GPS utilizado procede de la aeronave mediante MSDK, no de la ubicación Android del control.

4.4. Componentes Clave

- `PayloadProtocol.kt`: Decodificador con soporte dual para 32 B (v1) y 64 B (v2) con validación CRC32 y retiro de boots anteriores.
- `SampleHold.kt`: Zero-Order Hold que retiene las 9 variables simultáneamente a 10 Hz.
- `CsvJournal.kt`: Registro append-only de 24 columnas con `RandomAccessFile` y `fd.sync()`.
- `FloatingMonitor.kt`: Ventana flotante compacta arrastrable (`TYPE_APPLICATION_OVERLAY`) sobre DJI Pilot 2 con indicadores RX y BT.
- `BluetoothRelay.kt`: Servidor RFCOMM con canal conflado y watchdog de socket (3 s).

4.5. Pasos de Compilación y Variantes de APK

En Android Studio puedes seleccionar la variante en la pestaña **Build Variants** (abajo a la izquierda):
 * **demoDebug**: Genera `app/build/outputs/apk/demo/debug/app-demo-debug.apk` (para pruebas en mesa).
 * **djiDebug**: Genera `app/build/outputs/apk/dji/debug/app-dji-debug.apk` (para vuelo real con el RC Pro).

Desde la terminal:

```
cd ~/Documents/GasLab/MicroGas/app_dji

# Compilar simulador demo
./gradlew :app:assembleDemoDebug

# Compilar versión de vuelo real DJI
./gradlew :app:assembleDjiDebug

# Instalar en el control conectado por USB
adb install -r app/build/outputs/apk/demo/debug/app-demo-debug.apk
```

5. App 2: MicroGas Receptor para Celular Android (app_receptor)

5.1. Requisitos y Herramientas de Desarrollo

- **Compatibilidad**: Android 8.0 Oreo (API 26) o posterior según compatibilidad del dispositivo; `compileSdk 35` no establece una versión máxima.
- **Cero dependencias DJI**: Proyecto 100% estándar de Android con Jetpack Compose y Material 3.

5.2. Componentes Clave

- `BluetoothClient.kt`: Cliente RFCOMM con reconexión progresiva con backoff (5, 10, 20, 30 s), watchdog de 15 s por intento y descarte seguro.
- `NdjsonFramer` (en `Measurement.kt`): Buffer de tramas UTF-8 con límite de 16 KiB que soporta fragmentación de paquetes en la red.
- `MeasurementParser` (en `Measurement.kt`): Analizador tolerante a fallos que extrae las 9 variables físicas y las coordenadas GPS.
- `MeasurementRepository.kt`: Buffer en RAM de hasta **7.200 muestras** (~2 horas a 1 Hz), métricas de enlace (pérdida de secuencias, reinicios de sesión) y respaldo automático.
- `AlertManager.kt`: Vibración y notificación visual al cruzar el umbral configurable de CO₂ (ej. 3.000 ppm) con datos frescos (< 5 s).
- `Co2Chart.kt`: Gráfica interactiva de alto rendimiento implementada en Canvas nativo de Compose.

5.3. Interfaz de Usuario y Control de Zoom

1. Pestaña 1 (Monitor en tiempo real):

- Tarjeta principal con el valor actual de CO₂ en gran formato, indicador de frescura y origen (dji o SIMULADO).
- Selector de ventana de tiempo visible (1, 2, 5, 10, 15, 30 min).
- Gráfica en vivo de CO₂ vs tiempo.
- Tarjetas secundarias con Temperatura (°C) y Humedad (% HR).

2. Pestaña 2 (Histórico):

- Selector de ventana de tiempo (1, 2, 5, 10, 15, 30, 60 min o Todo).
- Nueve gráficas: CO₂, temperatura, humedad, PM1, PM2.5, PM4, PM10, VOC y NOx.
- **Control de Zoom en el Eje X:** El zoom táctil (*pinch-to-zoom*) y el desplazamiento (*pan*) se restringen al **eje temporal (X)** entre 1x y 60x con `.coerceIn()`, manteniendo la escala de concentración (Y) en autoajuste automático según los datos visibles.
- Tabla de datos crudos desplazable horizontal y verticalmente.
- Botón de exportación a CSV mediante Storage Access Framework (SAF).

5.4. Pasos de Compilación y Ubicación de APKs

¿Dónde se encuentra el APK generado?

- **APK Debug:** `app_receptor/app/build/outputs/apk/debug/app-debug.apk`
- **Release:** el receptor no configura una firma Release; usa Debug para esta prueba. No se promete un APK Release firmado.

Desde la terminal:

```
cd ~/Documents/GasLab/MicroGas/app_receptor

# Compilar APK Debug
./gradlew assembleDebug

# Ver el archivo generado
ls -lh app/build/outputs/apk/debug/app-debug.apk

# Instalar en el celular conectado por USB
adb install -r app/build/outputs/apk/debug/app-debug.apk
```

6. Guía de Integración y Prueba Extremo a Extremo

6.1. Prueba Local sin Hardware (Modo Simulado con 9 Variables)

Permite validar la sincronización completa entre dos dispositivos Android sin dron ni sensores: 1. Instalar `app-demo-debug.apk` en un dispositivo Android físico A. Un emulador sirve para UI, pero no verifica el enlace Bluetooth físico. 2. Instalar `app-debug.apk` en el celular B. 3. Emparejar ambos dispositivos por Bluetooth en los Ajustes del sistema. 4. En el dispositivo A, abrir MicroGas y pulsar **Iniciar pop-up o Segundo plano**. 5. En el celular B, abrir MicroGas Receptor y seleccionar el dispositivo A. 6. **Comprobación:** - El estado pasa a "Conectado". - En la pestaña Monitor se grafica el CO₂ en tiempo real. - En Histórico, las nueve curvas y la tabla se llenan con datos simulados; latitud, longitud y altura contienen datos sintéticos. - Probar exportar el archivo CSV y verificar que contiene las 16 columnas con datos válidos.

6.2. Prueba en Banco de Trabajo con Hardware Real

1. Validar primero el lector SEN66 y el esquema eléctrico de la placa concreta. Confirmar alimentación, pinout y niveles lógicos en la documentación del sensor y adaptador; esta copia no contiene un montaje eléctrico validado.

2. Preparar el enlace de la Pi al dron con el adaptador compatible y la configuración PSDK verificada para ese adaptador.
 3. Ejecutar en la Pi 5 el programa de adquisición que envía el paquete v2 de 64 bytes por PSDK.
 4. En el control DJI RC Pro Enterprise, abrir MicroGas DJI. Comprobar la recepción de paquetes MGAS y la validez del CRC32.
 5. Conectar el celular por Bluetooth al control y confirmar que las muestras recibidas incluyen la posición del dron asociada al recibir CO₂.
-

6.3. Prueba de convivencia con DJI Pilot 2

1. En el RC Pro Enterprise, iniciar MicroGas y pulsar **Iniciar pop-up**.
 2. Conceder el permiso “Mostrar sobre otras aplicaciones”.
 3. Pulsar **Ocultar app** para enviar la aplicación al fondo sin cancelar la adquisición.
 4. Abrir **DJI Pilot 2**:
 - Ubicar la ventana flotante (FloatingMonitor) donde permita operar los controles necesarios y verificarlo en el dispositivo. Las interacciones dentro de la ventana pertenecen a MicroGas.
 - RX ✓ indica recepción fresca, BT ✓ conexión y BT ↑ envío local. Verificar también que aumenten las muestras guardadas en el receptor.
 - Si la concentración supera el umbral configurado, el texto de CO₂ alterna blanco y rojo y se emiten los pitidos de advertencia.
-

7. Resolución de Problemas Frecuentes (Troubleshooting)

A. Desconexión al apagar la pantalla del celular

- Desde Receptor 0.4.2, ocultar la app o bloquear la pantalla ya no cierra Bluetooth. Un servicio con notificación mantiene recepción, reconexión y CSV.
- Verifica la notificación MicroGas Receptor activo. Desconectar detiene el servicio. El cierre forzado o la terminación del proceso interrumpen la recepción; no se recuperan automáticamente paquetes perdidos. El CSV del emisor es independiente.

B. Valores opcionales en blanco (“—”) o null

- **Causa:** El paquete PSDK recibido es v1 (32 bytes) o el sensor se encuentra en ciclo inicial de calentamiento (*warm-up*).
- **Comportamiento:** La app sigue operando normalmente y grafica el CO₂ sin interrupción.

C. Error de CRC32 en la recepción

- **Causa:** Desajuste en el algoritmo de cálculo de CRC o en el endianness entre el código C de la Raspberry Pi y Java/Kotlin.
- **Solución:** Usar el polinomio estándar reflected 0xEDB88320 con valor inicial y XOR final 0xFFFFFFFF (CRC-32/ISO-HDLC), idéntico al implementado por `java.util.zip.CRC32`.

GasLab

*doc-
u-
mentación
de
im-
ple-
mentación;
las
prue-
bas
de
hard-
ware
se
reg-
is-
tran
por
sep-
a-
rado.
##*

8. In-
cor-
po-
ración:
em-
pezar
desde
una
com-
puta-
dora
nueva

Esta
sec-
ción
está
pen-
sada
para
una
per-
sona
que
recibe
la
car-
peta
y
aún
no
conoce
el
proyecto.
Primero
re-
pro-
duce
la
sim-
u-
lación
y el
análi-
sis;
de-
spués
inte-
gra
el
hard-
ware.
Una
com-
pi-
lación
cor-
recta
no
de-
mues-
tra
fun-
cionamiento
del
sen-
sor,
GPS
real
ni
con-
viven-
cia
con

8.1.
Ubi-
cación
de
cada
componente

text
GasLab/
Mi-
cro-
Gas/
MAN-
UAL_REPLICACION_DESDE_CERO.md
app_dji/

proyecto
Gra-
dle,
vari-
antes
DJI
y
Sim-
u-
lado
app_receptor/

otro
proyecto
Gra-
dle
in-
de-
pen-
di-
ente
apks/

copias
de
en-
trega
grafi-
ca-
cionCom-
pleta/

anal-
izador
Python
de
CSV
mini-
hawk/

proyecto
in-
de-
pen-
di-
ente
MAVLink
PCBs
GasLAB/
#12
dis-
eños

Abre
cada
proyecto
An-
droid
por
sep-
a-
rado.
Mi-
cro-
Gas/
no
tiene
un
wrap-
per
Gra-
dle
con-
junto.
Mini-
Hawk
usa
otros
pro-
toco-
los y
sen-
sores;
su
firmware
no
susti-
tuye
al
emisor
PSDK
de
Mi-
cro-
Gas.
Los
mód-
du-
los
app_dji/microgas_desktop/
son
lóg-
ica
de
pro-
to-
colo/transporte,
no
una
in-
ter-
faz
de
es-

En
Linux,
adapta
una
sola
ruta
para
los
co-
man-
dos
siguientes:
bash
ex-
port
GASLAB_DIR="/home/cristopher/Documents/GasLab"

8.2.
En-
torno
Android

Instala
JDK
17,
An-
droid
Stu-
dio o
las
her-
ramien-
tas
An-
droid,
SDK
Plat-
form
35 y
Plat-
form
Tools
(ADB).
Usa
ini-
cial-
mente
las
ver-
siones
fi-
jadas
en el
proyecto:
Gra-
dle
8.9,
AGP
8.7.3,
Kotlin
2.0.21
y
Com-
pose
BOM
2024.12.01.
No
actu-
al-
ices
to-
das
las
de-
pen-
den-
cias
mien-
tras
in-
ten-
tas
re-

```
bash
java
-
version
adb
ver-
sion
cd
"$GASLAB_DIR/MicroGas/app_dji"
./gradlew
--
version
```

Gradle
debe
usar
Java
17.
An-
droid
Stu-
dio y
la
ter-
mi-
nal
pueden
tener
JDK
dis-
tin-
tos:
re-
visa
Gra-
dle
JDK
en el
IDE.
Cada
app
necesita
su
pro-
pio
lo-
cal.properties
con
sdk.dir
apun-
tando
al
SDK
de
esta
com-
puta-
dora.
An-
droid
Stu-
dio
puede
gener-
arlo.
No
copies
ese
archivo
desde
otra
com-
puta-

La
primera
sin-
cronización
re-
quiere
in-
ter-
net
para
descar-
gar
de-
pen-
den-
cias.
--
offline
solo
sirve
de-
spués
de
ten-
erlas
en
caché.
Si
fal-
tan
per-
misos
del
wrap-
per
usa
chmod
+x
gradlew
desde
su
car-
peta.
En
Win-
dows
uti-
liza
gradlew.bat
y
adapta
los
co-
man-
dos
del
shell.

Para
con-
figu-
rar
la
cre-
den-
cial,
con-
sulta
tam-
bién
el
[ini-
cio
del
README
DJI](#).
Este
proyecto
no
lee
un
campo
APP_ID:
re-
quiere
la
App
Key
MSDK.

La
vari-
ante
DJI
lee
la
clave
de
DJI_MSDK_API_KEY
o, si
no
está
definida,
de
dji.msdk.apiKey
en
lo-
cal.properties.
La
clave
cor-
re-
sponde
al
pa-
quete
com.gaslab.microgas.
Puede
com-
pilar
sin
ella,
pero
no
ini-
ciará
re-
cep-
ción
real.
Demo
y Re-
cep-
tor
no la
nece-
si-
tan.
No
com-
par-
tas
cre-
den-
ciales
ni
con-
fun-
das
la
clave

```
###
8.3.
Com-
pilar
y
preparar
los
tres
instaladores
bash
cd
"$GASLAB_DIR/MicroGas/app_dji"
./gradlew
:app:testDemoDebugUnitTest
:app:testDjiDebugUnitTest
./gradlew
:app:lintDemoDebug
:app:lintDjiDebug
./gradlew
:app:assembleDemoDebug
:app:assembleDjiDebug
cd
"$GASLAB_DIR/MicroGas/app_receptor"
./gradlew
:app:testDebugUnitTest
:app:lintDebug
:app:assembleDebug
```

Comprueba

BUILD

SUC -

CESS -

FUL

y re-

visa

in-

formes

en

app/build/reports/.

Los

con-

teos

es-

critos

en

los

README

son

históri-

cos:

para

una

en-

trega

repite

los

co-

man-

dos

y an-

ota

el re-

sul-

tado.

Si

de-

spués

de

mover

car-

petas

apare-

cen

ref-

eren-

cias

no

re-

sueatas

a

fun-

ciones

exis-

tentes,

sin-

croniza

y22

prueba

re-

Gradle
no
actu-
aliza
au-
tomáti-
ca-
mente
las
copias
de
en-
trega.
De-
spués
de
com-
pillar
correctamente:
bash
cd
"\$GASLAB_DIR/MicroGas"
mkdir
-p
apks
cp
app_dji/app/build/outputs/apk/demo/debug/app-
demo-
debug.apk
apks/MicroGas-
Simulado-
debug.apk
cp
app_dji/app/build/outputs/apk/dji/debug/app-
dji-
debug.apk
apks/MicroGas-
DJI-
debug.apk
cp
app_receptor/app/build/outputs/apk/debug/app-
debug.apk
apks/MicroGas-
Receptor-
debug.apk
sha256sum
apks/*.apk

```

|
|
| APK
|
| Iden-
| tifi-
| cador
| An-
| droid
| Uso
| |—
| —
| —| |
| Sim-
| u-
| lado
|
| com.gaslab.microgas.demo
|
| Emisor
| sin-
| tético,
| An-
| droid
| 10+
|
| DJI |
| com.gaslab.microgas
|
| Emisor
| real
| con
| MSDK,
| An-
| droid
| 10+,
| arm64
| |
| Re-
| cep-
| tor |
| com.gaslab.microgas.receptor
|
| Cliente
| Blue-
| tooth,
| An-
| droid
| 8+ |

```

Release
de
DJI
usa
ac-
tual-
mente
la
firma
De-
bug
de
Gra-
dle.
Re-
cep-
tor
no
con-
figura
firma
Re-
lease.
Es-
tos
in-
stal-
adores
De-
bug
son
para
prue-
bas;
una
en-
trega
de
pro-
duc-
ción
re-
quiere
su
propia
con-
figu-
ración
de
firma
y
cus-
to-
dia
de
clave.

8.4.
In-
sta-
lar y
re-
cono-
cer
el
dispositivo
Activa
depu-
ración
USB,
conecta
el
dis-
posi-
tivo
y
acepta
la
au-
tor-
ización
de la
com-
puta-
dora
en
su
pantalla:
bash
adb
de-
vices
-l
Debe
apare-
cer
de-
vice,
no
unau-
tho-
rized.
Si
hay
var-
ios,
usa
su
se-
rial
explícitamente:

```
_____  
bash  
cd  
"$GASLAB_DIR/MicroGas"  
adb  
-s  
SE-  
RIAL_EMITOR  
in-  
stall  
-r  
apks/MicroGas-  
Simulado-  
debug.apk  
adb  
-s  
SE-  
RIAL_RECEPTOR  
in-  
stall  
-r  
apks/MicroGas-  
Receptor-  
debug.apk  
adb  
-s  
SE-  
RIAL_CONTROL  
in-  
stall  
-r  
apks/MicroGas-  
DJI-  
debug.apk
```

Sustituye
los
seri-
ales;
el
ter-
cer
co-
mando
cor-
re-
sponde
a la
etapa
real.
- r
con-
serva
datos
si la
firma
es
com-
pati-
ble.
Ante
in-
com-
pati-
bili-
dad
de
firma,
ex-
porta
CSV
antes
de
desin-
sta-
lar:
los
archivos
pri-
va-
dos
se
elim-
inan
al
desin-
sta-
lar o
bor-
rar
datos.
Demo
y DJI
pueden
es-
tar

9.
Prueba
guiada
y re-
sul-
ta-
dos
esperados

Usa
dos
An-
droid
fisi-
cos,
sin
dron:
Sim-
u-
lado
en A
y Re-
cep-
tor
en B.
Em-
pareja
desde
los
ajustes
Blue-
tooth,
con-
cede
per-
misos
y
pulsa
**Se-
gundo
plano**
en A
para
ini-
ciar
servi-
dor y
adquisi-
ción
sin
per-
miso
de
over-
lay.
En B
se-
lec-
ciona
A;
desde
Re-
cep-
tor
0.4.2
puedes
ocul-
tarlo
y
man-

Espera
un
min-
uto.
Deben
cre-
cer
el
con-
ta-
dor
CSV
del
emisor
y las
mues-
tras
del
re-
cep-
tor.
Abre
Histórico,
toca
una
mues-
tra y
prueba
zoom/paneo.
Ex-
porta
CSV
en
am-
bos
equipos:
deben
tener
24 y
16
colum-
nas
re-
spec-
tiva-
mente.
El
emisor
suele
tener
más
filas
porque
repite
la úl-
tima
mues-
tra a
~30
Hz;
eso

Detén
A y
ob-
serva
rein-
ten-
tos.
Reiní-
cialo
y
veri-
fica
re-
cep-
ción
y
cam-
bio
de
sesión.
El
en-
lace
no
recu-
pera
ret-
ro-
spec-
tiva-
mente
lo
per-
dido.
Cierra
y
reabre
B:
las
gráfi-
cas
en
memo-
ria
pueden
perderse,
pero
Archivos
CSV
debe
con-
ser-
var
los
respaldos.

9.1.
Colores,
alarma
y
pausa
El
umbral
es
independiente
en
cada
dispositivo.
Para
probar
configura
700
ppm
en
ambos:
la
demo
oscila
aproximadamente
entre
490
y
810
ppm.
Después
restaura
el
valor
de
trabajo.

—
| Lec-
tura
re-
specto
al
um-
bral
T |
Texto
CO₂
| |—
|—| |
Menor
que
70 %
|
Blanco
| |
Desde
70 %
hasta
90 %
in-
clu-
sive
|
Amar-
illo |
|
Mayor
que
90
%,
sin
alarma
ac-
tiva |
Rojo
| |
Alarma
ac-
tiva
desde
T in-
clu-
sive
| Al-
terna
blanco/rojo
cada
500
ms |

Con
T =
700,
amar-
illo
comienza
en
490,
rojo
por
encima
de
630
y
alarma
desde
700.
La
alarma
necesita
datos
frescos
de
hasta
5 segundos.
DJI
emite
pitidos
desde
su
servicio;
Receptor
vibra
al
entrar
y
muestran
banner
mientras
está
visible y
conectado.
La
última
toma
lectura

**Pausar
vista**

en el
emisor

no
de-
tiene

CSV,
Blue-
tooth

ni
piti-
dos.

El
texto
con-

ge-
lado
no

parpadea
como

lec-
tura
ac-

tual.

**Se-
gundo
plano**

del
pop-
up

solo
oculta
la

ven-
tana;

**De-
tener**

sí
final-
iza

adquisi-
ción.

La
barra
vi-

sual
es
rela-

tiva
al
máx-

imo
ob-
ser-

vado,
no al
umbral.

9.2.
Se-
lec-
cionar
un
punto

Receptor

per-
mite
to-
car
pun-
tos
en
Mon-
itor
y en
sus
nueve
gráfi-
cas
históri-
cas.

Se
se-
lec-
ciona
una
mues-
tra
cer-
cana
en
pan-
talla,
no
un
valor
in-
ter-
po-
lado.

La
tar-
jeta
aparece
de-
bajo
con
hora
hasta
milise-
gun-
dos,
valor,
CO₂
y co-
orde-
nadas
si
exis-
ten.

Seis
deci-
males

de
GPS

El
zoom
y pa-
neo
del
histórico
lle-
gan
a
60x.
**Cer-
rar
de-
talle,**
un
toque
lejos
de
los
pun-
tos o
**Restable-
cer**
qui-
tan
la se-
lec-
ción.
Al
salir
la
mues-
tra
de la
ven-
tana
visi-
ble
deja
de
mostrarse
su
de-
talle.
DJI y
Sim-
u-
lado
tienen
tabla
histórica,
no
es-
tas
nueve
gráficas.

10.
Com-
pren-
der
el
reg-
istro
antes
de
analizarlo

10.1.
Mues-
tras,
filas
y
relojes

|
Campo
| In-
ter-
pretación
| |—
|—| |
fecha_hora_utc
en
emisor
|
Hora
de
es-
crit-
ura
de la
fila
CSV
| |
sen66_muestra_utc
| Re-
cep-
ción
de la
mues-
tra
en el
con-
trol,
no
adquisi-
ción
física
en el
sen-
sor |
|
sen66_new_data
| 1 si
es
pos-
te-
rior
a la
úl-
tima
mues-
tra
guardada
cor-
rec-
ta-
mente;
0 si
es
retenida
| |
sen66_secuencia
|
Iden-

Las
fechas
CSV
son
UTC.
La
hora
en
pan-
talla
de-
pende
de la
zona
del
dis-
posi-
tivo.
El
GPS
cor-
re-
sponde
a la
re-
cep-
ción
de
CO₂;
no
ex-
iste
sin-
cronización
que
per-
mita
afir-
mar
que
es la
posi-
ción
ex-
acta
en el
in-
stante
de
adquisi-
ción
en la
Pi.

DJI
acepta
GPS
de
hasta
5 se-
gun-
dos,
coor-
de-
nadas
fini-
tas
váli-
das
y
nivel
de
señal
3-5
o 10.
El
nivel
10
no
certi-
fica
pre-
cisión
RTK.
Si
falta
GPS,
con-
tinúa
CO₂
y las
coor-
de-
nadas
quedan
vacías.
No
se
usan
la
ubi-
cación
An-
droid
del
con-
trol
ni
ceros
de
relleno.

10.2.
Ex-
portación
correcta

DJI
con-
serva
hasta
3.600
mues-
tras
en
memo-
ria y
Re-
cep-
tor
hasta
7.200.
Sus
archivos
per-
sis-
tentes
no
es-
tán
limi-
ta-
dos
a
esas
can-
ti-
dades.
**Ex-
por-
tar
CSV**
del
histórico
re-
cep-
tor
ex-
porta
la
memo-
ria;
**Archivos
CSV**
→
**Ex-
por-
tar**
copia
el
respaldo
per-
sis-
tente.
Para
se⁴⁵
siones
largas

El
se-
lec-
tor
An-
droid
per-
mite
USB,
car-
peta
u
otros
provee-
dores
disponibles.
Can-
celar
con-
serva
el
origi-
nal;
un
fallo
puede
de-
jar el
des-
tino
in-
com-
pleto.
Rein-
tenta
y re-
visa
los
con-
ta-
dores
de
es-
crit-
ura.
Ex-
porta
antes
de
rein-
sta-
lar o
bor-
rar
datos.

Usa
nom-
bres
de
columna
al
analizar,
no
posi-
ciones.
Emisor
uti-
liza
ori-
gen;
re-
cep-
tor
ori-
gin.
Los
CSV
an-
tiguos
se
ex-
por-
tan
sin
trans-
for-
mar
su
es-
quema.
Op-
cionales
ausentes
quedan
vacíos,
no
son
cero.
Blue-
tooth
no
garan-
tiza
recu-
perar
cortes
ni
con-
fir-
mar
que
cada
es-
crt-
ura
del

```
##
11.
Del
CSV
a
una
grá-
fica
y un
mapa
Desde
la
raíz
del
es-
pa-
cio
de
trabajo:
bash
cd
"$GASLAB_DIR/graficacionCompleta"
python3
-m
venv
.venv
.venv/bin/python
-m
pip
in-
stall
-r
re-
quire-
ments.txt
.venv/bin/python
-m
unittest
-v
test_graficar.py
./ini-
ciar.sh
```

Requiere
Python
3.10+
y Tk-
inter.
En
Ubuntu,
si fal-
tan,
in-
stala
python3-
venv
y
python3-
tk.
Crea
el
en-
torno
en
cada
com-
puta-
dora;
no
copies
.venv.
Para
de-
talles
con-
sulta
el
[README](#)
del
grafi-
cador.

Abre
un
CSV
y se-
lec-
ciona
vari-
ables
con
Ctrl/Shift.
fecha
mues-
tra
fecha/hora;
hora
mantiene
los
cam-
bios
de
día;
tran-
scur-
rido
usa
se-
gun-
dos
desde
la
primera
fecha
vál-
ida;
times-
tamp
mues-
tra
se-
gun-
dos
Unix;
in-
dice
no
re-
quiere
fechas.
La
zona
ini-
cial
es
Amer-
ica/Costa_Rica;
fechas
sin
zona
sẽ
in-
ter-

Para
estu-
diar
mues-
tras
DJI
nuevas
ac-
tiva
Solo
mues-
tras
nuevas
SEN66
y
usa
sen66_muestra_utc.
Para
estu-
diar
es-
crit-
ura
con-
serva
to-
das
las
filas
y
usa
fecha_hora_utc.
En
re-
cep-
tor
usa
fecha_hora_utc.
Cada
vari-
able
tiene
es-
cala
propia
y se
puede
guardar
con
la
barra
Matplotlib.

**Ruta
sin
in-
ter-
net**

dibuja
lati-
tud/longitud.

**Mapa
satelital**

gen-
era
HTML

y
abre
el
nave-
g-
ador;
nece-
sita
in-
ter-
net
para
las
bib-
liote-
cas
y el
fondo.

No
se
sube
el
CSV
como
archivo,
pero
el
HTML
con-
tiene
las
coor-
de-
nadas
y los
ser-
vi-
cios
de
ma-
pas
reciben
solic-
i-
tudes
de⁵²
imá-
genes.

El
mapa
corta
la
ruta
ante
GPS
in-
válido
y
cam-
bios
de
ses-
sion,
cuando
esa
columna
ex-
iste.
No
cal-
cula
dis-
tan-
cias
geodési-
cas
ni
elim-
ina
saltos
GPS
váli-
dos.
Si
con-
cate-
nas
vue-
los
sin
iden-
tifi-
cadores
de
sesión
puede
unir-
los:
anal-
iza
los
archivos
por
sep-
a-
rado.
La⁵³
ruta
con-

```
Prueba
sin
vuelo
real,
con
coord-
de-
nadas
sin-
teti-
cas
in-
clu-
idas
en
ejem-
plo.csv:
bash
cd
"$GASLAB_DIR/graficacionCompleta"
mkdir
-p
ejem-
plos
.venv/bin/python
graficar.py
ejem-
plo.csv
--y
co2_ppm
tem-
per-
atura_c
--
eje
hora
--
salida
ejem-
p-
los/sensores.png
--
ruta
ejem-
p-
los/ruta.png
--
mapa
ejem-
p-
los/ruta.html
```

Debes
obtener
dos
imá-
genes
y un
HTML.
La
curva
cruza
me-
di-
anoche
y la
ruta
tiene
un
hueco
por
GPS
fal-
tante.
La
demo
An-
droid
y
este
archivo
de
ejem-
plo
gen-
eran
coor-
de-
nadas
sin-
téti-
cas
para
pro-
bar
mapas.

12.
In-
cor-
po-
rar
hard-
ware
y tra-
ba-
jar
en el
código

12.1.
Lo
que
falta
apor-
tar
para
replicar
el
montaje

Este
es-
pa-
cio
in-
cluye
apps,
con-
tratos
y se-
rial-
izador,
pero
no
un
firmware
com-
pleto
de la
Pi,
un
driver
inte-
grado
del
sen-
sor
ni un
es-
quema
eléc-
trico
vali-
dado
del
pay-
load.
El
inte-
grador
debe
disponer
de
sen-
sor/placa,
ali-
mentación
y
adap-
ta-
dor
ade-
cua-
dos,
proyecto
Payload-
SDK
ex-
terno
y su
con-

Valida
por
eta-
pas:
lec-
tura
nueva
y
CRC
del
sen-
sor;
seri-
al-
ización
local
C;
en-
vío
PSDK;
reg-
istro
MSDK
y
puerto
UP;
lle-
gada
de
mues-
tras
a An-
droid;
GPS;
Blue-
tooth;
ex-
portación
y
con-
viven-
cia
con
Pilot
2 en
banco.
Guarda
evi-
den-
cia
de
cada
etapa.
Una
trama
vál-
ida
en
un
test
no

boot_id
cam-
bia
en
cada
ar-
ranque;
se-
quence
au-
menta
por
mues-
tra
nueva
aunque
el
CO₂
no
cam-
bie.
No
re-
generes
iden-
ti-
dad
al
reen-
viar.
Usa
Mi-
cro-
Gas_SendMeasurement
para
v2;
Mi-
cro-
Gas_SendCo2
con-
serva
v1.
El
CRC
del
pa-
quete
no
reem-
plaza
el
CRC
del
sen-
sor.
El
de-
talle
esta
en el
con-

12.2.
Dónde
mod-
ificar
cada
comportamiento

—
| Tra-
bajo
|
Punto
ini-
cial |
|—
|—| |
Sim-
u-
lación
| DJI
Sam-
ple-
Source.kt
||
Re-
cep-
ción
real
y
GPS
| DJI
src/dji/.../DjiSampleSource.kt,
Dji-
Air-
craft-
Po-
si-
tion.kt
||
Vali-
dación
bina-
ria |
DJI
Pay-
load-
Pro-
to-
col.kt
||
Adquisi-
ción
y al-
ma-
ce-
namiento
| DJI
Mon-
i-
torView-
Model.kt
(in-
cluye
Mon-
i-
torEngine),
61
Sam-
ple-
Hold.kt.

Los
archivos
Kotlin
prin-
ci-
pales
viven
bajo
app/src/main/java/com/gaslab/microgas/
o, en
re-
cep-
tor,
su
sub-
pa-
quete
re-
cep-
tor/.
Las
im-
ple-
menta-
ciones
ex-
clusi-
vas
de
DJI
viven
bajo
app/src/dji/.
No
con-
fun-
das
clase
y
archivo:
algu-
nas
clases
com-
parten
archivo.

Antes
de
añadir
una
vari-
able
recorre
C →
de-
codi-
fi-
cador
→
mod-
elo
→
CSV
→
JSON
→ re-
cep-
tor
→
gráfi-
cas.
De-
fine
unidades,
nu-
los y
com-
pati-
bili-
dad.
Actu-
aliza
las
ref-
eren-
cias
sen66-
v2.hex
y
sen66-
v2.ndjson
solo
ante
un
cam-
bio
in-
ten-
cional
del
con-
trato.
con-
texto_app.md
63
con-
serva
prop-

12.3.
Diag-
nós-
tico
y
entrega

| Síntoma
|
Primera
com-
pro-
bación
| |—
|—| |
No
ex-
iste
gradlew
| En-
tra
al
proyecto
An-
droid,
no a
GasLab
o Mi-
cro-
Gas
| |
SDK
no
en-
con-
trado
|
sdk.dir
y
SDK
Plat-
form
35
de
esa
máquina
| |
Of-
fline
no
re-
suelve
de-
pen-
den-
cias
|
Primera
com-
pi-
lación
con
red |
|
65
MSDK
es-
pera

Para
re-
por-
tar
fal-
los
in-
cluye
vari-
ante,
equipo,
ac-
ción,
men-
saje
lit-
eral,
fecha
y
ejem-
plo
de
CSV.
Puedes
guardar
adb
-s
SE-
RIAL
log-
cat
-d >
log-
cat.txt.
No
rein-
stales
bor-
rando
datos
como
primera
medida.

En
cada
en-
trega
reg-
istra
fecha,
ver-
sión
declarada,
re-
visión
del
código
si ex-
iste,
co-
man-
dos
de
prueba,
hashes
de
APK
y
qué
se
probó
físi-
ca-
mente.
Con-
serva
CSV
origi-
nal y
re-
sul-
ta-
dos
deriva-
dos
por
sep-
a-
rado.
Para
campo
an-
ota
tam-
bién
ver-
siones
de
firmware,
du-
ración,
un-
bral,
es-

Antes
de
tra-
ba-
jar
por
tu
cuenta
debes
poder
ex-
plicar
qué
hace
cada
APK,
com-
pi-
larlo,
conec-
tar
Demo
con
Re-
cep-
tor,
ex-
por-
tar
un
respaldo
per-
sis-
tente,
in-
ter-
pre-
tar
los
relo-
jes,
generar
una
grá-
fica
y
señalar
qué
parte
del
mon-
taje
aún
falta
integrar.

Prueba
de
ruta
y al-
tura
(0.4.0)

1.
Actu-
aliza
Sim-
u-
lado
y Re-
cep-
tor.
Ini-
cia
la
adquisi-
ción
y
conecta
am-
bos
por
Blue-
tooth.

2.
En
Sim-
u-
lado
veri-
fica
las
colum-
nas
lati-
tud,
lon-
gi-
tud
y al-
tura:
la al-
tura
ini-
cia
en 0
m y
varía
hasta
60
m;
las
coor-
de-
nadas
son
sin-
tétic-
as.

3.
Abre
Ruta
**y al-
tura**

La
ruta
es
una
proyec-
ción
local,
sin
imá-
genes
car-
tográ-
ficas.
El re-
cep-
tor
no
fab-
rica
GPS
cuando
falta:
con-
serva
hue-
cos
y
usa
hasta
7.200
mues-
tras
en
memoria.

Se-
lec-
ción
vin-
cu-
lada
y
botones
(0.4.1)

- En
DJI/Simulado,

**Ocul-
tar**

app

aparece

antes

de

**De-
tener,**

en

su

an-

tigua

posi-

ción.

Com-

pro-

bar

que

Ocul-

tar

en-

vía

la

ac-

tivi-

dad

al

fondo

y

con-

serva

el

mon-

itor;

De-

tener

sigue

dete-

niendo

la

adquisi-

ción.

- En

Mon-

itor

o

Histórico

del

re-

cep-

tor,

to-

car

un

punto

y

72
luego

el

bloque

Re-
cep-
ción
en
se-
gundo
plano
(Re-
cep-
tor
0.4.2)

Prueba
física
pen-
di-
ente:
conec-
tar a
Sim-
u-
lado,
ano-
tar
con-
ta-
dor y
nom-
bre
CSV,
ir a
Ini-
cio,
blo-
quear
la
pan-
talla
var-
ios
min-
utos
y
volver.
Deben
con-
ser-
varse
la
sesión,
el
nom-
bre
CSV
y las
mues-
tras
recibidas
du-
rante
ese
in-
ter-
valo.
Repe-
tir
qui-
tando
la
ac-
ti-
dad
de

Se
usó
el
tipo
An-
droid
con-
nected-
De-
vice.
No
se
ha
im-
ple-
men-
tado
ACK
ni re-
trans-
misión
de
mues-
tras
perdidas.

13. Guía de Desarrollo con Android Studio y Extensión de Features

Esta sección está diseñada para desarrolladores, investigadores o estudiantes que deseen modificar la aplicación o agregar nuevas funciones, **incluso si nunca antes han utilizado Android Studio**.

13.1. Cómo funciona Android Studio (Guía para principiantes)

1. Regla de oro: Cómo abrir el proyecto

- En Android Studio, ve a **File → Open**.
- **NUNCA abras la carpeta raíz MicroGas/ completa.** Android Studio se confundirá porque contiene dos proyectos Gradle independientes.
- Abre la carpeta específica con la que vas a trabajar:
 - Abre /home/.../MicroGas/app_dji/ para trabajar en la app del control remoto o simulador.
 - Abre /home/.../MicroGas/app_receptor/ para trabajar en la app del celular de monitoreo.

2. La ventana de Android Studio y sus vistas En el panel lateral izquierdo verás la estructura del proyecto. Arriba de ese panel hay un menú desplegable con opciones de vista: * **Vista Android (Recomendada para programar):** Oculta archivos de configuración complejos y te muestra solo lo que necesitas: * app / manifests / AndroidManifest.xml: Aquí se declaran los permisos (Bluetooth, Notificaciones, etc.) y las pantallas/servicios de la app. * app / java / com.gaslab.microgas...: Aquí está todo el código fuente en lenguaje **Kotlin** (.kt). * app / res /: Recursos gráficos, iconos de la app y temas de colores. * Gradle Scripts: Los archivos build.gradle.kts que controlan las librerías externas y la versión de compilación. * **Vista Project:** Muestra la estructura de carpetas real de tu disco duro tal como la verías en el explorador de archivos.

3. El botón del Elefante (Sync Project with Gradle Files) Gradle es el sistema que descarga librerías y compila el código. Cada vez que abras el proyecto por primera vez, o si modificas un archivo `build.gradle.kts`, verás una barra amarilla arriba o un icono de un **elefante con una flecha azul** en la esquina superior derecha. Haz clic en él para sincronizar las dependencias.

4. Build Variants (Solo en app_dji) En `app_dji` existen dos variantes de compilación (*Flavors*):
* `demoDebug`: No necesita dron ni hardware DJI. Simula la generación de gases y ruta.
* `djiDebug`: Conecta con el DJI Mobile SDK real para comunicarse con la aeronave.
* Para cambiar entre una y otra: haz clic en la pestaña **Build Variants** (abajo a la izquierda) y en la columna *Active Build Variant* selecciona `demoDebug` o `djiDebug`.

5. Cómo probar la app en tu teléfono o en el control

1. En tu teléfono Android, ve a *Ajustes* → *Información del teléfono* y pulsa 7 veces seguidas sobre *Número de compilación* para activar el menú de desarrollador.
2. Ve a *Ajustes* → *Opciones de desarrollador* y activa **Depuración por USB (USB Debugging)**.
3. Conecta el dispositivo a la computadora por cable USB. En el teléfono aparecerá un aviso: “¿Permitir depuración por USB?”, pulsa **Permitir**.
4. En la barra superior de Android Studio, verás un selector de dispositivos donde aparecerá el modelo de tu teléfono.
5. Haz clic en el botón verde de **Play** (icono de triángulo verde o atajo Shift + F10). Android Studio compilará la app, la instalará en el dispositivo y la abrirá automáticamente.

6. ¿Dónde ver qué está pasando y cómo detectar errores?

- **Pestaña Logcat (Barra inferior):** Es la consola de depuración en vivo. Puedes ver mensajes del sistema y llamadas `println(...)`. Si la aplicación se cierra inesperadamente (*Crash*), escribe `fatal` o `Exception` en la barra de búsqueda de Logcat; te mostrará en **texto rojo** el error exacto y el archivo y número de línea que causó el fallo.
- **Pestaña Build (Barra inferior):** Te avisa si hay errores de sintaxis o tipos antes de ejecutar la app.

7. Entendiendo la tecnología: Jetpack Compose Estas aplicaciones **no utilizan los antiguos archivos XML de diseño**. Toda la interfaz se construye con **Jetpack Compose**:
* Cada elemento visual (un botón, una tarjeta, una gráfica) es una función de Kotlin marcada con la anotación `@Composable`.
* **Estado reactivo:** La interfaz no se manipula manualmente (no existe `findViewById` ni `button.setText`). En *Receptor*, *ReceiverEngine* conserva la sesión y publica el estado (*StateFlow*); *MonitorViewModel* lo expone a *Compose* y *ReceiverService* mantiene la recepción. En *DJI*, *MonitorEngine*, definido en `MonitorViewModel.kt`, gestiona adquisición y almacenamiento. Cuando llega una nueva muestra de gas del sensor o de Bluetooth, el estado cambia y *Compose* redibuja automáticamente los componentes necesarios.

13.2. Mapa de desarrollo: ¿Qué archivo modifico o creo para cada feature?

Si deseas extender o personalizar el sistema, consulta esta guía rápida de archivos según el tipo de cambio que quieras realizar:

Caso 1: Quiero agregar una nueva pantalla o pestaña en la app de monitoreo

1. **Crear el diseño:** En `app_receptor/app/src/main/java/com/gaslab/microgas/receptor/`, crea un archivo Kotlin (ej. `NuevaPantalla.kt`) y define tu función `@Composable` `fun NuevaPantalla(state: MonitorUiState) { ... }`.

2. **Agregar la pestaña:** Abre MainActivity.kt. En el bloque TabRow(selectedTabIndex = ...), añade el nombre de tu pestaña a la lista: listOf("Monitor", "Histórico", "Ruta y altura", "Nueva Pestaña").
3. **Renderizarla:** En el bloque condicional when (state.selectedTab) o if (state.selectedTab == 3) dentro de MainActivity.kt, invoca tu función NuevaPantalla(state, ...).

Caso 2: Quiero agregar una nueva variable física o sensor

1. **En el receptor (Modelo y parseo):**
 - Abre Measurement.kt: Añade la nueva propiedad al data class Measurement(..., val miVariable: Float? = null).
 - En el mismo archivo, dentro de MeasurementParser.parse(), lee el dato del objeto JSON recibido por Bluetooth: val miVar = json finite("mi_variable")?.toFloat().
2. **En las pantallas:**
 - En HistoryScreen.kt: Añade una gráfica Co2Chart(state.history.samples, "Mi Variable (unidad)", ..., value = { it.miVariable }) y agrega una columna en la tabla de datos crudos.
3. **En el respaldo CSV:**
 - En MeasurementRepository.kt y CsvArchive.kt: Añade el nombre de la columna al encabezado CSV y el valor en la fila generada.
4. **En el emisor (app_dji):**
 - En PayloadProtocol.kt: Decodifica el nuevo campo si viene en la trama binaria del sensor.
 - En LiveTelemetry.kt: Agrégalo al JSON que se transmite por Bluetooth: \ "mi_variable\" :
....

Caso 3: Quiero modificar los colores, umbrales o alarmas

1. **Reglas de color (Blanco, Amarillo, Rojo, Parpadeo):**
 - Abre Co2Appearance.kt (existe uno en cada proyecto). Modifica la función co2ReadingColor(co2, threshold, alarm, whitePhase).
2. **Lógica de activación de alarma:**
 - En Receptor, AlertManager.kt define isCo2AlertActive, invocada desde ReceiverEngine.refreshAlert(). En DJI, LiveTelemetry.kt define LiveTelemetry.alarm y LiveTelemetry.fresh; MonitorService.kt las usa para controlar los pitidos y el overlay. Revisa sus pruebas al cambiar fresca o umbral.
3. **Diálogo de ajuste de umbral:**
 - En MainActivity.kt de app_receptor busca thresholdDialog para cambiar el rango permitido (ej. 1 a 100.000 ppm) o el valor predeterminado.

Caso 4: Quiero modificar el mapa de ruta o las gráficas

1. **Gráficas de líneas y puntos táctiles:**
 - Abre Co2Chart.kt: Contiene el dibujo en Canvas (drawPath, drawCircle), las líneas de cuadrícula y la lógica de gestos de zoom y paneo (detectTransformGestures).
 - Abre ChartSelection.kt: Contiene el algoritmo euclidiano nearestChartPoint que busca la muestra más cercana al dedo cuando el usuario toca la pantalla.
2. **Lógica de ruta y posición:**
 - Revisa RouteScreen.kt (Canvas, gestos, Home, selector y detalle), RouteGeometry.kt (proyección e inversa para ejes), RouteHighlights.kt (métricas y clasificación) y MeasurementRepository.kt (Home y máximos de toda la sesión). Las coordenadas se manejan en pares (sample.latitude, sample.longitude).

Caso 5: Quiero modificar la comunicación Bluetooth

1. **Emisor (Control DJI):**
 - BluetoothRelay.kt: Inicia el socket servidor RFCOMM usando el UUID bb239920-bdbc-4d51-8a12-a5351875d891.

- LiveTelemetry.kt: Serializa los datos en una línea de texto JSON (formato NDJSON terminado en \n).

2. Receptor (Celular):

- BluetoothClient.kt: Maneja el socket cliente, el hilo de lectura y los reintentos automáticos progresivos (5s, 10s, 20s, 30s).
- NdjsonFramer (dentro de Measurement.kt): Acumula los bytes recibidos del flujo y los corta en líneas completas cuando encuentra un salto de línea \n.

Caso 6: Quiero modificar la ventana flotante (Overlay sobre DJI Pilot 2)

1. **Diseño de la ventana:** Abre FloatingMonitor.kt. Controla el tamaño de la ventana, la vista minimizada (solo número de CO₂ y estado) y la vista expandida con arrastre táctil.
2. **Servicio en segundo plano:** Abre MonitorService.kt. Mantiene la notificación, los pitidos y el overlay. La solicitud del permiso de superposición (SYSTEM_ALERT_WINDOW) se inicia desde MonitorControls.kt con ACTION_MANAGE_OVERLAY_PERMISSION.

13.3. Compilación automatizada mediante Makefile (make apks)

Para facilitar el trabajo en la terminal sin depender de abrir Android Studio cada vez que se requiera compilar un instalador, se ha integrado un Makefile en la raíz del repositorio (/MicroGas/):

```
# Compilar las 3 variantes y copiarlas automáticamente a la carpeta ./apks/
make apks

# 0 compilar variantes individuales:
make dji      # Genera apks/MicroGas-DJI-debug.apk (MSDK v5)
make demo     # Genera apks/MicroGas-Simulado-debug.apk (Simulador)
make receptor # Genera apks/MicroGas-Receptor-debug.apk (Celular de monitoreo)

# Ejecutar todas las pruebas unitarias automáticas:
make test

# Ejecutar análisis estático:
make lint

# Limpiar archivos temporales de compilación:
make clean
```

Este comando asegura que los APKs resultantes queden centralizados en la carpeta apks/ con nombres limpios y listos para transferir a los dispositivos.

14. Receptor 0.4.3: Home, máximos y administración de CSV

- **Home:** icono verde en la primera posición GPS válida recibida de la sesión; no certifica el punto de despegue. Se conserva aunque la muestra salga del buffer.
- **10 valores más altos:** selector de las nueve variables; mantiene las 10 muestras de mayor valor de toda la sesión de recepción, sin límite de antigüedad ni dependencia de la ventana visible o del buffer de 7.200 muestras. Al llegar datos se actualiza la clasificación; en empates se conserva primero la muestra recibida antes. Los valores ausentes/no finitos se excluyen. Con menos de 10 valores válidos se muestran los disponibles.
- Los máximos con GPS se resaltan en magenta sobre el mapa, incluidos los antiguos. La lista ordenada permite seleccionar cada máximo; los que no tienen GPS conservan su puesto y muestran «sin GPS». La trayectoria y el perfil de altura siguen usando el historial reciente. Home y máximos se reinician al iniciar manualmente otra sesión, no al reconectar automáticamente ni al cambiar la sesión del emisor. No se reconstruyen tras terminar el proceso.
- **Ejes geográficos:** marcas de latitud y longitud sobre la cuadrícula, actualizadas con zoom y paneo; proyección local con norte arriba.

- **CSV:** la lista rápida permite exportar. Para borrar entra en **Archivos CSV → Administrar CSV → Eliminar... → Borrar definitivamente**. Cada archivo requiere confirmación y la sesión activa permanece protegida; desconecta antes de borrarla.
- **Makefile:** `make apks`, `make dji`, `make demo` y `make receptor` compilan y copian los instaladores a `apks/`; `make test` ejecuta pruebas y `make lint` el análisis estático. `make clean` limpia las compilaciones. Las tareas se ejecutan en serie para evitar compilar simultáneamente el mismo proyecto.

Prueba funcional de la nueva versión

1. Conecta Simulado y Receptor en dos Android autorizados para depuración. Abre **Ruta y altura**: comprueba Home verde y las marcas de latitud/longitud al ampliar y desplazar el mapa.
2. Cambia la variable en **Máximos**. Comprueba el orden descendente, selecciona un punto de la lista y verifica su detalle y resaltado. Si falta GPS, la muestra debe permanecer en la clasificación sin crear una posición.
3. En una sesión larga, verifica que un máximo temprano y Home siguen disponibles después de 7.200 muestras; el perfil y la trayectoria reciente conservan su límite de memoria. Inicia manualmente otra sesión y comprueba el reinicio de Home y máximos.
4. Exporta un CSV desde la lista rápida. Entra en **Administrar CSV**, cancela un borrado y comprueba que el archivo continúa; confirma el borrado de un archivo de prueba. El CSV activo no debe poder eliminarse mientras la recepción esté activa.

La implementación pasó 36 pruebas unitarias del receptor, 47 por variante del emisor y lint sin errores. `make apks` generó los tres APK y sus firmas se verificaron. Estos resultados no sustituyen la prueba visual/Bluetooth anterior: el dispositivo disponible estaba sin autorización ADB. Consulta [el informe](#).

El Makefile limita Gradle a dos workers y 1024 MB de heap y evita tareas paralelas sobre un mismo proyecto. Puedes añadir opciones mediante `GRADLE_FLAGS`; al sustituirlo conserva las opciones de recursos si el equipo tiene memoria limitada. `--offline` solo funciona si las dependencias ya están descargadas.

15. Cómo editar menús e interfaz en Android Studio

Esta sección explica paso a paso cómo agregar o modificar elementos de la interfaz del receptor (botones, diálogos, pestañas) y cómo conectarlos con la lógica interna. Los ejemplos usan el código real de `app_receptor`.

Importante: Esta app usa **Jetpack Compose**. No hay archivos XML de layout. Toda la interfaz es código Kotlin. No existe un editor de arrastrar y soltar como en el diseño XML tradicional. Los cambios de UI se hacen editando funciones `@Composable` directamente en el código.

15.1. Orientarse en Android Studio y encontrar el archivo correcto

Abrir el proyecto correcto

1. Android Studio → **Open** → selecciona la carpeta `app_receptor/` (no la carpeta raíz `MicroGas/`).
2. Espera a que termine el **Gradle Sync** (barra de progreso en la parte inferior de la pantalla).
3. En el panel izquierdo, cambia el selector de vista de **Android** a **Project** para ver todos los archivos reales del disco.

Estructura de archivos relevante

```

app_receptor/
└─ app/src/main/java/com/gaslab/microgas/receptor/
    └─ MainActivity.kt      ← barra superior, botones, diálogos, pestañas
    └─ MonitorScreen.kt    ← contenido de la pestaña "Monitor"
    └─ HistoryScreen.kt    ← contenido de la pestaña "Histórico"
    └─ RouteScreen.kt      ← contenido de la pestaña "Ruta y altura"
    └─ MonitorViewModel.kt ← lógica interna (sin UI); expone state y funciones

```

Encontrar rápidamente dónde está algo Usa **Ctrl+Shift+F** (buscar en todo el proyecto). Si ves el texto “Administrar CSV” en la app, búscalo y Android Studio te lleva exactamente a la línea donde está definido. Es la forma más rápida de orientarse.

15.2. Editar la GUI: botones, diálogos y pestañas

Todos los ejemplos siguientes corresponden a código real en MainActivity.kt.

Ejemplo A — Agregar un botón en la barra superior Los botones **Bluetooth** y **Umbral** están en las líneas ~99-100 de MainActivity.kt:

```

// Código actual
TextButton(onClick = { connectionDialog = true; refreshWithPermission() }) { Text("Bluetooth") }
TextButton(onClick = { thresholdDialog = true }) { Text("Umbral") }

```

Para **agregar un botón nuevo** al lado de “Umbral”, simplemente añades otro TextButton en la misma Row:

```

TextButton(onClick = { connectionDialog = true; refreshWithPermission() }) { Text("Bluetooth") }
TextButton(onClick = { thresholdDialog = true }) { Text("Umbral") }
TextButton(onClick = { /* tu acción aquí */ }) { Text("Mi botón") } // ← nuevo

```

No necesitas declarar el botón en ningún otro lugar. Compose detecta el cambio y lo muestra automáticamente al compilar.

Ejemplo B — Agregar una opción dentro de un diálogo existente El diálogo de **Archivos CSV** está alrededor de la línea 139 de MainActivity.kt. Dentro del bloque que genera la fila de cada archivo (items(state.csvFiles)), el Row actual tiene solo el botón **Exportar**:

```

// Código actual
Row {
    TextButton(onClick = { /* exportar */ }) { Text("Exportar") }
}

```

Para agregar un segundo botón en la misma fila:

```

Row {
    TextButton(onClick = { /* exportar */ }) { Text("Exportar") }
    TextButton(onClick = { /* nueva acción */ }) { Text("Compartir") } // ← nuevo
}

```

Ejemplo C — Agregar un diálogo nuevo (desde cero) Un diálogo en Compose sigue siempre el mismo patrón de dos partes:

Parte 1 — Variable de estado que controla si el diálogo está abierto (se declara con el resto de variables al inicio de ReceptorApp):

```

var miDialogo by rememberSaveable { mutableStateOf(false) }

```

Parte 2 — El diálogo en sí (se agrega al final de ReceptorApp, fuera del Column principal):

```

if (miDialogo) AlertDialog(
    onDismissRequest = { miDialogo = false },
    title = { Text("Mi nuevo diálogo") },
    text = { Text("Contenido del diálogo") },
    confirmButton = { TextButton(onClick = { miDialogo = false }) { Text("Aceptar") } },
    dismissButton = { TextButton(onClick = { miDialogo = false }) { Text("Cancelar") } },
)

```

Parte 3 — El botón que lo abre (en cualquier lugar de la UI):

```
TextButton(onClick = { miDialogo = true }) { Text("Abrir mi diálogo") }
```

Este es exactamente el mismo patrón que usan los diálogos de Umbral, Bluetooth y Archivos CSV ya existentes.

Ejemplo D — Agregar una pestaña nueva Paso 1: En la línea ~119 de MainActivity.kt, agrega el nombre al final de la lista:

```

// Antes (3 pestañas)
listOf("Monitor", "Histórico", "Ruta y altura").forEachIndexed { index, name ->

// Después (4 pestañas)
listOf("Monitor", "Histórico", "Ruta y altura", "Mi pestaña").forEachIndexed { index, name ->

```

Paso 2: En el bloque condicional ~línea 123, agrega la nueva condición al inicio:

```

if (state.selectedTab == 3) {
    MiNuevaPantalla(modifier.weight(1f)) // función @Composable que defines tú
} else if (state.selectedTab == 2) {
    RouteScreen(...)
} else if (...) {
    ...
}

```

Paso 3: Crea el archivo de la pantalla nueva. En la misma carpeta que MainActivity.kt, crea MiNuevaPantalla.kt:

```

package com.gaslab.microgas.receptor

import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier

@Composable
fun MiNuevaPantalla(modifier: Modifier = Modifier) {
    Text("Hola desde mi nueva pestaña")
}

```

15.3. Editar la lógica interna: ViewModel y estado

La regla de esta app es que **la UI no hace lógica directamente**. En cambio, llama funciones del ViewModel:

```

Acción del usuario (toque de botón)
→ la UI llama vm.miFuncion()
→ MonitorViewModel ejecuta la lógica
→ actualiza _state con .update { it.copy(...) }
→ Compose detecta el cambio y redibuja solo lo necesario

```

Paso a paso para agregar lógica nueva 1. Agregar el dato al estado — busca data class MonitorUiState en MonitorViewModel.kt y añade el campo:

```
data class MonitorUiState(
    val threshold: Int = 3000,
    val miNuevoDato: String = "", // ← nuevo campo con valor inicial
    // ...
)
```

2. Agregar la función en el ViewModel — dentro de la clase MonitorViewModel:

```
fun actualizarMiDato(valor: String) {
    // aquí va la lógica (validación, cálculo, etc.)
    _state.update { it.copy(miNuevoDato = valor) }
}
```

3. Llamarla desde la UI — en cualquier pantalla que tenga acceso al ViewModel:

```
// En MainActivity.kt, MonitorScreen.kt, etc.
TextButton(onClick = { vm.actualizarMiDato("nuevo valor") }) {
    Text("Hacer algo")
}
```

```
// Para mostrar el dato:
Text("Estado: ${state.miNuevoDato}")
```

Compose se encarga automáticamente de redibujar el Text cuando miNuevoDato cambia, porque el estado se observa con collectAsStateWithLifecycle().

15.4. Previsualizar cambios sin instalar en el teléfono

Compose permite ver el resultado visual de un @Composable directamente en Android Studio sin compilar ni conectar ningún dispositivo, usando la anotación @Preview.

```
import androidx.compose.ui.tooling.preview.Preview

@Preview(showBackground = true, backgroundColor = 0xFF1A1A1A)
@Composable
fun PreviewMiBoton() {
    MaterialTheme {
        TextButton(onClick = {}) { Text("Mi botón nuevo") }
    }
}
```

Al agregar esta función en cualquier archivo Kotlin del proyecto, Android Studio muestra un ícono de ojo en el margen izquierdo junto a la anotación @Preview. Al hacer clic se abre el panel **Design** a la derecha con la vista renderizada en vivo.

Cuándo sirve el Preview: - Para ajustar colores, tamaños, padding y textos sin compilar. - Para prototipar la apariencia de un nuevo botón o diálogo.

Cuándo no sirve: - No ejecuta lógica de negocio real (ViewModel, Bluetooth, CSV). - Para probar comportamiento hay que instalar en el teléfono con **Shift+F10** o el botón de triángulo verde.

15.5. Flujo de trabajo recomendado

1. Identificar el archivo
 - Busca el texto visible en la app con Ctrl+Shift+F
 - Te lleva directamente a la línea exacta
2. Editar el Composable
 - Agregar el elemento de UI (botón, diálogo, pestaña)
 - Ajustar con @Preview si quieres ver el resultado visual
3. Agregar lógica si hace falta
 - Nuevo campo en MonitorUiState

→ Nueva función en MonitorViewModel

4. Compilar y probar en el teléfono

- Ctrl+F9 para compilar sin instalar (detecta errores)
- Shift+F10 o triángulo verde para instalar y ejecutar

5. Verificar en el teléfono

- Comprobar que el elemento aparece y funciona
- Rotar pantalla para verificar que no se rompe el layout

Atajos útiles de Android Studio:

Atajo	Acción
Ctrl+Shift+F	Buscar texto en todo el proyecto
Ctrl+B / Ctrl+Click	Ir a la definición de una función o clase
Shift+F10	Compilar e instalar en el dispositivo conectado
Ctrl+F9	Solo compilar (sin instalar), detecta errores de sintaxis
Alt+Enter	Sugerencia de autocorrección / importar clase faltante
Ctrl+Z	Deshacer el último cambio
Ctrl+Shift+Z	Rehacer